

Table des matières

1) Problématique.....	1
2) Exercices	1
3) Conclusion.....	15

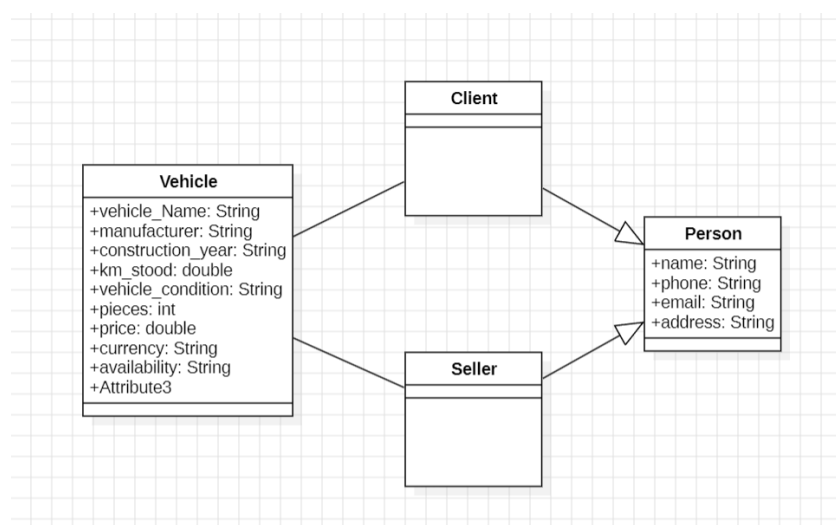
1) Problématique

Nous souhaitons créer une application permettant à un garage de gérer ses véhicules, ses vendeurs, les ventes et ses clients. Pour cela nous allons créer un diagramme de classe et l'utiliser pour générer un code Java et une base de données SQL qui seront utilisés par la suite pour créer l'application en utilisant en plus de cela XAMPP, JDBC et SceneBuilder.

2) Exercices

Question 1 : Diagramme de classe, code Java et SQL.

Pour cette question je vais utiliser l'essai gratuit du logiciel StarUML pour créer le diagramme de classe, voici à quoi il ressemble (héritage de person par client et seller et association des deux à vehicle) :

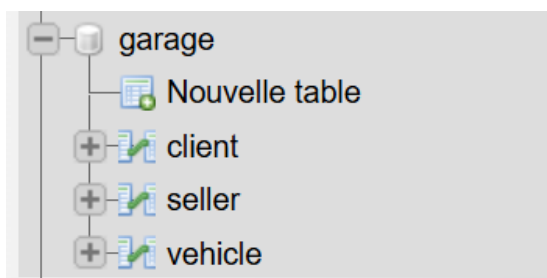


Dans StarUML pour générer du code Java il faut tout d'abord installer les bibliothèques dans la section Tools > Extensions puis chercher Java, l'installer et redémarrer le logiciel. Ensuite pour générer le code Java il faudra aller dans Tools > Java > Generate Code. Le résultat devrait être le suivant :

 Client.java	28/04/2026 13:17	Fichier JAVA	1 Ko
 Person.java	28/04/2026 13:17	Fichier JAVA	1 Ko
 Seller.java	28/04/2026 13:17	Fichier JAVA	1 Ko
 Vehicle.java	28/04/2026 13:17	Fichier JAVA	1 Ko

Lorsque le Java est créé il ne reste plus qu'à créer les tables SQL, malheureusement StarUML ne propose que du SQL Postgre donc nous allons devoir créer les tables manuellement.

Le résultat devrait être le suivant :



Question 2 :

Pour pouvoir créer notre application, nous devons créer getters et setters manuellement, et placer les fichiers .java dans un même dossier que l'on appellera « cardealer », il y aura aussi le main (« HelloApplication ») et les controllers dans celui-ci :

CarDealer2 > src > main > java > com > example > cardealer			
Nom	Modifié le	Type	Taille
 Client.java	28/04/2026 13:35	Fichier JAVA	1 Ko
 HelloApplication.java	28/04/2026 13:36	Fichier JAVA	2 Ko
 HelloController.java	28/04/2026 13:36	Fichier JAVA	1 Ko
 Person.java	28/04/2026 13:35	Fichier JAVA	2 Ko
 Seller.java	28/04/2026 13:35	Fichier JAVA	1 Ko
 Vehicle.java	28/04/2026 13:36	Fichier JAVA	3 Ko

Ensuite dans autre dossier nous allons créer notre fxml dans un autre dossier « cardealer » présent dans un dossier « ressources » :

hello-view.fxml

28/04/2026 13:27

Fichier FXML

7 Ko

Question 3 :

Lorsque le FXML est créé on peut commencer à créer la maquette. Pour le menu on va d'abord créer un borderpane dans lequel nous allons mettre une VBox qui contiendra une menu bar avec 3 menus dedans (Main menu, Overview, Deal). Ensuite on ajoute une anchor pane en dessous de la borderpane et on ajoute 4 image view dans lesquels nous allons ajouter les images données dans le zip avec les icons de voitures, ainsi qu'un label vert nommé « Welcome to your car dealer ». Le résultat devrait être le suivant :



Question 4 :

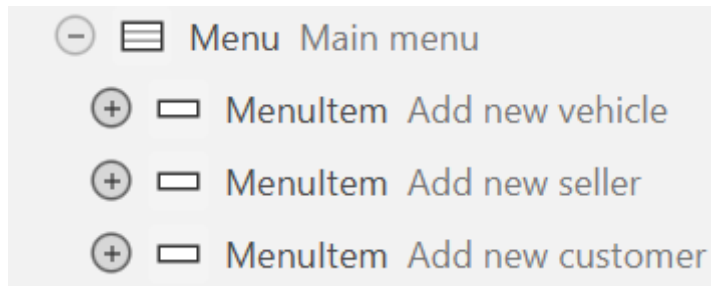
Pour modifier la menubar nous allons avant ça rajouter les options d'ajouts de voiture, de clients et de vendeurs dans le controller :

```
@FXML
protected void onAddVehicle() {
    System.out.println("Add new vehicle clicked");
}

@FXML
protected void onAddSeller() {
    System.out.println("Add new seller clicked");
}

@FXML
protected void onAddCustomer() {
    System.out.println("Add new customer clicked");
}
```

Ensuite dans SceneBuilder on va dans le main menu dans le menu bar et on ajoute trois menu items comme ceci :



Question 5 :

On souhaite créer un menu pour l'ajout de véhicules, pour cela on va créer un nouveau fichier FXML nommé `add_vehicles.fxml`, et un controller pour celui-ci nommé `NewVehicle.java`, ensuite on modifie le `HelloController` pour faire le lien avec la page de base et le menu d'ajout de véhicules.

Lorsque c'est fait sur SceneBuilder on ajoute une anchor pane et une gridpane, dans l'anchorpane on ajoute une vbox qui contiendra l'image du vehicule et le label, dans la gridpane il y aura des labels suivis de text fields et tout en bas on ajoute une Hbox avec 3 boutons, le résultat final devrait ressembler à ça :



Question 6 : Déjà faite dans la question précédente.

Question 7 :

On va désormais lier notre java au sql, pour cela on a déjà le mysql connector donc il suffit simplement de créer une classe que l'on appellera DBConnection, ensuite pour faire marcher le code qui nous est donné on va enlever le mdp en modifiant le « mdp » en “ ”. Il faudra aussi créer une bdd nommé javafx-course :

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class DBConnection {

    public static Connection getConnection() throws SQLException {
        try {
            // Charge le pilote JDBC MySQL
            Class.forName("com.mysql.cj.jdbc.Driver");
        } catch (ClassNotFoundException e) {
            e.printStackTrace();
            throw new SQLException("MySQL JDBC driver not found", e);
        }

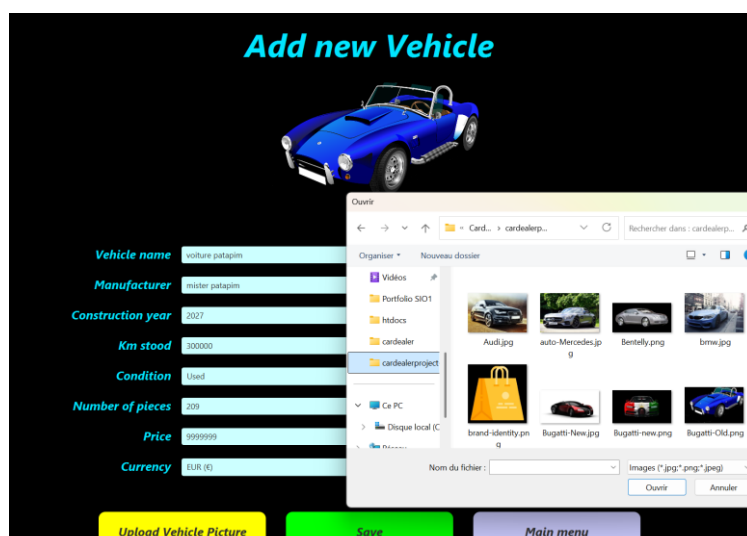
        // Retourne la connexion établie à la base de données
        // REMPLACE "votre_mdp" par ton vrai mot de passe MySQL (laisse vide "" si tu n'en as pas)
        return DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/javafx-course",
            "root",
            ""
        );
    }
}
```

Exercice 8 :

Pour pouvoir ajouter l'option d'ajouts d'image nous ajoutons la bibliothèque File Chooser : import javafx.stage.FileChooser;

On ajoute aussi le FileInputStream qui servira de mémoire.

On modifie le controller NewVehicle pour pouvoir faire marcher les IDs et les lier au nouveau code de NewVehicle, cela devrait marcher après enregistrement :

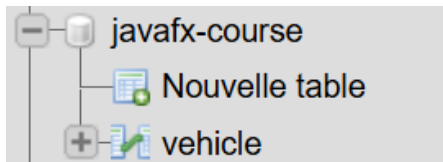


(au passage j'ai du refaire un dossier où j'ai tout mis dedans car c'était compliqué avec ce qui était demandé de base)

Exercice 9

Maintenant on va faire fonctionner le bouton save.

Pour cela le code va récupérer les données des textfields, les lire et les enregistrer dans une table SQL nommé « vehicle » :



En cours de route j'ai eu un souci a priori il n'est pas possible d'envoyer de données supérieures à 1Mo à xampp donc il faut cliquer sur le config à côté de MySQL dans XAMPP, aller sur my.ini et modifier la ligne de max_allowed_packet pour changer la valeur de 1M avec 16M.

```
max_allowed_packet=16M
sort_buffer_size=512K
net_buffer_length=8K
read_buffer_size=256K
read_rnd_buffer_size=512K
myisam_sort_buffer_size=8M
log_error="mysql_error.log"
```

Maintenant que c'est corrigé on peut utiliser le bouton save et ça marche :

Add new Vehicle



Vehicle name:

Manufacturer:

Construction year:

Km stood:

Condition:

Number of pieces:

Price:

Currency:

	id	name	manufacturer	year	km	condition	pieces	price	currency	image
☐ Éditer ☐ Copier ☐ Supprimer	1	voiture	monsieur	1999	1	New	10000	5	EUR (€)	[BLOB - 724,3 kio]

Exercice 10 :

Pour cet exercice c'est simple il suffit de faire en sorte de charger la page du menu principal lorsqu'on clique sur le bouton « Main Menu » on crée donc une méthode exprès :

```
@FXML
void onMainMenu(ActionEvent event) {
    try {
        FXMLLoader loader = new FXMLLoader(getClass().getResource("hello-view.fxml"));
        Parent root = loader.load();
        Stage stage = (Stage) myanchorpane.getScene().getWindow();
        stage.setScene(new Scene(root, 1200, 900));
    } catch (IOException e) {
        e.printStackTrace();
    }
}
```

Exercice 11 :

On va créer la page avec le TableView, pour cela on va tout d'abord créer une nouvelle classe nommée « VehicleTable.java » (controller), tout d'abord on modifie la table Vehicle créé précédemment pour faire marcher le code :

```
public class Vehicle {
    private int id; // Ajouté pour correspondre à ta clé primaire SQL
    private String vehicleName;
    private String manufacturer;
    private String constructionYear;
    private double kmStood;
    private String vehicleCondition;
    private int pieces;
    private double price;
    private String currency;
    private String availability;
    private byte[] image;

    // Constructeur vide
    public Vehicle() {}

    // Constructeur
    public Vehicle(int id, String vehicleName, String manufacturer, String constructionYear,
        double kmStood, String vehicleCondition, int pieces,
        double price, String currency, String availability, byte[] image) {
        this.id = id;
        this.vehicleName = vehicleName;
        this.manufacturer = manufacturer;
        this.constructionYear = constructionYear;
        this.kmStood = kmStood;
        this.vehicleCondition = vehicleCondition;
        this.pieces = pieces;
        this.price = price;
        this.currency = currency;
        this.availability = availability;
        this.image = image;
    }

    // Getters
    public int getId() { return id; }
    public String getVehicleName() { return vehicleName; }
    public String getManufacturer() { return manufacturer; }
    public String getConstructionYear() { return constructionYear; }
    public double getKmStood() { return kmStood; }
    public String getVehicleCondition() { return vehicleCondition; }
    public int getPieces() { return pieces; }
    public double getPrice() { return price; }
    public String getCurrency() { return currency; }
    public String getAvailability() { return availability; }
    public byte[] getImage() { return image; }
}
```

Ensuite on crée un vehicletable.fxml et on adapte pour que ça ressemble au résultat suivant :

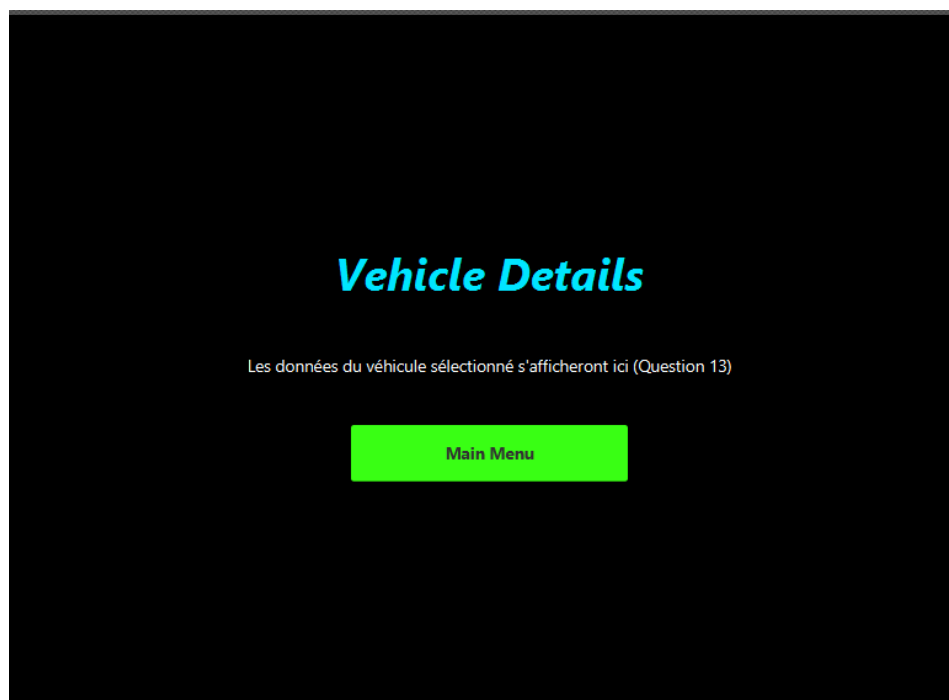
Exercice 12 :

Désormais, nous souhaitons pouvoir faire apparaître notre vehicule dans un nouveau layout avec un clique de souris sur la tableview, pour cela on va d'abord créer un nouveau controller nommé « SelectedVehicle » :

```
1 import javafx.fxml.FXML;
2 import javafx.fxml.FXMLLoader;
3 import javafx.scene.Parent;
4 import javafx.scene.Scene;
5 import javafx.stage.Stage;
6 import javafx.event.ActionEvent;
7 import java.io.IOException;
8
9 public class SelectedVehicle {
10
11     @FXML
12     protected void onMainMenu(ActionEvent event) {
13         try {
14             FXMLLoader loader = new FXMLLoader(getClass().getResource("hello-view.fxml"));
15             Parent root = loader.load();
16             Stage stage = (Stage) ((javafx.scene.Node) event.getSource()).getScene().getWindow();
17             stage.setScene(new Scene(root, 1100, 750));
18             stage.setTitle("Main Menu");
19             stage.show();
20         } catch (IOException e) {
21             e.printStackTrace();
22         }
23     }
24 }
```

Et modifie VehicleTable.java pour prendre en charge l'affichage et le chargement des données de la table vehicle dans notre BDD.

Et biensûr, vu que nous avons créé un controller il faut faire un fxml associé, donc on crée un fichier Selectedvehicle.fxml :



(affichage temporaire, on le change à la question 13)

Question 13 et 14 :

Maintenant qu'on sait à quoi doit ressembler le fxml de l'affichage, on modifie l'ancien, ainsi que le controller pour que tout fonctionne correctement :

```
import javafx.fxml.FXML;
import javafx.fxml.FXMLLoader;
import javafx.fxml.Initializable;
import javafx.scene.Parent;
import javafx.scene.Scene;
import javafx.scene.control.TextField;
import javafx.scene.image.Image;
import javafx.scene.image.ImageView;
import javafx.stage.Stage;
import javafx.event.ActionEvent;
import java.io.ByteArrayInputStream;
import java.io.IOException;
import java.net.URL;
import java.util.ResourceBundle;

public class SelectedVehicle implements Initializable {

    @FXML private ImageView iv_selectedVehicle;
    @FXML private TextField tf_id, tf_name, tf_manufacturer, tf_year, tf_km, tf_condition, tf_pieces, tf_price, tf_currency, tf_availability;

    @Override
    public void initialize(URL url, ResourceBundle rb) {
        // Au chargement, on remplit les champs avec les variables statiques de VehicleTable
        tf_id.setText(String.valueOf(VehicleTable.id_number));
        tf_name.setText(VehicleTable.vehicle_name);
        tf_manufacturer.setText(VehicleTable.manufacturer);
        tf_year.setText(VehicleTable.construction_year);
        tf_km.setText(String.valueOf(VehicleTable.kmstood));
        tf_condition.setText(VehicleTable.condition);
        tf_pieces.setText(String.valueOf(VehicleTable.pieces));
        tf_price.setText(String.valueOf(VehicleTable.price));
        tf_currency.setText(VehicleTable.currency);
        tf_availability.setText(VehicleTable.availability);

        // Affichage de l'image si elle existe
        if (VehicleTable.imageBytes != null) {
            iv_selectedVehicle.setImage(new Image(new ByteArrayInputStream(VehicleTable.imageBytes)));
        }
    }

    @FXML
    protected void onTableVehicle(ActionEvent event) {
        try {
            FXMLLoader loader = new FXMLLoader(getClass().getResource("vehicletable.fxml"));
            Parent root = loader.load();
            Stage stage = (Stage) tf_id.getScene().getWindow();
            stage.setScene(new Scene(root, 1100, 750));
            stage.setTitle("Vehicle Table");
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Résultat :

Update Vehicle Data



Vehicle ID	1
Vehicle name	voiture
Manufacturer	monsieur
Construction year	1999
Km stood	1.0
Condition	New
Number of pieces	10000
Price	5.0
Currency	EUR (€)
Availability	In Stock

Upload PictureSave PictureUpdateDeleteTable Vehicle

Exercice 15 :

Nous souhaitons faire fonctionner le bouton « upload picture » donc pour cela nous allons ajouter une variable encapsulée « path » qui servira de mémoire temporaire et la fonction onUploadPicture() qui servira à laisser l'utilisation modifier l'image actuel :



Code :

```
@FXML
protected void onUploadPicture(ActionEvent event) {
    FileChooser fileChooser = new FileChooser();
    fileChooser.setTitle("Upload Vehicle Picture");
    fileChooser.getExtensionFilters().addAll(
        new FileChooser.ExtensionFilter("Image Files", "*.jpg", "*.png", "*.jpeg")
    );

    Stage stage = (Stage) tf_id.getScene().getWindow();
    File file = fileChooser.showOpenDialog(stage);

    if (file != null) {
        try {
            path = file.getAbsolutePath();
            iv_selectedVehicle.setImage(new Image(new FileInputStream(path)));
        } catch (FileNotFoundException e) {
            e.printStackTrace();
        }
    }
}
```

Cependant, l'image peut être modifiée mais elle n'est pas changée dans la base de données, il faut donc faire marcher le bouton save picture qui permettra donc de modifier l'image blob dans la bdd avec la nouvelle qui a été choisie, et c'est justement le sujet de l'exercice 16.

Exercice 16 :

Pour faire marcher le bouton save, rien de plus simple, il suffit de créer une fonction onSavePicture() qui sera utilisé dans SceneBuilder dans la partie On Action pour permettre de téléverser l'image choisi avec upload picture à la BDD et de sauvegarder le changement, comme ceci :

	id	name	manufacturer	year	km	condition	pieces	price	currency	image
 Éditer  Copier  Supprimer	1	voiture	monsieur	1999	1	New	10000	5	EUR (€)	[BLOB - 19,9 kio]

(le blob était de 140 ko avant)

Exercice 17 :

Maintenant nous devons nous occuper du bouton delete, le but de ce bouton est de pouvoir supprimer le véhicule sélectionné de la VehicleTable après confirmation. Encore une fois il suffit de créer une fonction (onDeleteVehicle) et de faire le lien dans SceneBuilder.

Code :

```
@FXML
protected void onDeleteVehicle(ActionEvent event) {
    Alert alert = new Alert(Alert.AlertType.CONFIRMATION);
    alert.setTitle("Warning");
    alert.setHeaderText("Delete Vehicle");
    alert.setContentText("Are you sure? Do you really want to delete this vehicle?");

    Optional<ButtonType> result = alert.showAndWait();

    if (result.isPresent() && result.get() == ButtonType.OK) {
        String sql = "DELETE FROM vehicle WHERE id = ?";

        try (Connection con = DBConnection.getConnection();
            PreparedStatement pstmt = con.prepareStatement(sql)) {

            pstmt.setInt(1, Integer.parseInt(tf_id.getText()));
            pstmt.executeUpdate();

            onTableVehicle(event);
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Résultat (après suppression) :

ID	Vehicle name	Manufacturer	Year	Km stood	Condition	Pieces	Price	Currency	Availability
Aucun contenu dans la table									

Exercice 18 :

Finalement, il ne reste plus que le bouton d'update, ce bouton permettra de modifier les textfields et de pouvoir enregistrer après (le bouton update se modifiera pour devenir un bouton confirm).

Résultats :

Avant update :

Vehicle ID	2
Vehicle name	voiture trop cool
Manufacturer	gars trop cool

Après :

Vehicle ID	2
Vehicle name	voiture trop génial
Manufacturer	gars trop génial

BDD :

	id	name	manufacturer	year	km	condition	pieces	price	currency	image
☐ Éditer ☐ Copier ☐ Supprimer	2	voiture trop génial	gars trop génial	1800	10000000	New	20000	20	USD (\$)	[BLOB - 305,4 kio]

Exercice 19 :

Il s'agit de la dernière ligne droite, il faut faire les mêmes étapes mais avec Seller et Customer.

Créations des tables :

```
CREATE TABLE seller (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(255) NOT NULL,  
    phone VARCHAR(50),  
    email VARCHAR(255),  
    address VARCHAR(255),  
    image LONGBLOB  
);
```

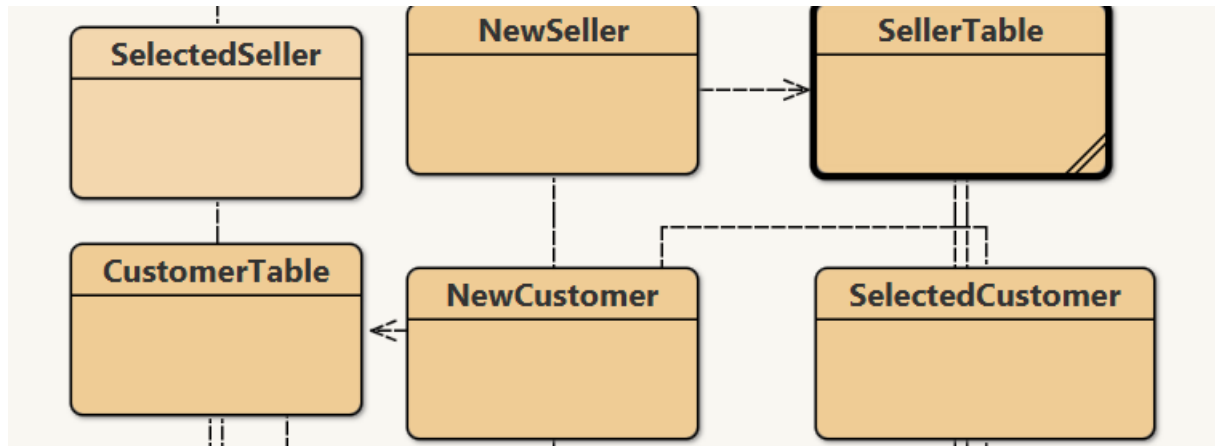
```
CREATE TABLE customer (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(255) NOT NULL,  
    phone VARCHAR(50),  
    email VARCHAR(255),  
    address VARCHAR(255),  
    image LONGBLOB  
);
```

Modifications classes seller et customer :

```
public class Seller extends Person {  
    private int id;  
    private byte[] image;  
  
    public Seller(int id, String name, String phone, String email, String address, byte[] image) {  
        super(name, phone, email, address);  
        this.id = id;  
        this.image = image;  
    }  
  
    public int getId() { return id; }  
    public void setId(int id) { this.id = id; }  
  
    public byte[] getImage() { return image; }  
    public void setImage(byte[] image) { this.image = image; }  
}
```

```
public class Customer extends Person {  
    private int id;  
    private byte[] image;  
  
    public Customer(int id, String name, String phone, String email, String address, byte[] image) {  
        super(name, phone, email, address);  
        this.id = id;  
        this.image = image;  
    }  
  
    public int getId() { return id; }  
    public void setId(int id) { this.id = id; }  
  
    public byte[] getImage() { return image; }  
    public void setImage(byte[] image) { this.image = image; }  
}
```

Création des controllers pour chaque fonctionnalités des clients et des vendeurs :



Créations des 6 fxml :

 selectedcustomer.fxml	05/05/2026 14:56	Fichier FXML	4 Ko
 customertable.fxml	05/05/2026 14:56	Fichier FXML	2 Ko
 newcustomer.fxml	05/05/2026 14:56	Fichier FXML	4 Ko
 selectedseller.fxml	05/05/2026 14:55	Fichier FXML	4 Ko
 sellertable.fxml	05/05/2026 14:55	Fichier FXML	2 Ko
 newseller.fxml	05/05/2026 14:55	Fichier FXML	4 Ko

3) Conclusion

En conclusion ce TP m'a permis de créer une application entière, répond à un besoin d'une organisation, dans le but de me faire comprendre toutes les démarches à faire et toutes les choses à comprendre lorsqu'on se lance dans ce genre de projet. C'était un projet amusant selon moi.